

Learning notes for LLM

Bingjin Liu

January 7, 2026

Abstract

This is the learning/summary notes for large language models (LLM). The references are the book [Speech and Language Processing](#) [1] and [LLM course by hugging face](#).

1 Tokenization

Tokenization is to convert text inputs to numerical data. There are several different methods:

1. word-based

The vocabulary can become very big. Also there are many similar words, for example “dog” and “dogs”. They are represented differently in word-based tokenization.

2. character-based

For Latin language, the character-based tokenization is less meaningful; since individual character does not mean much on its own. Another challenge is that the model needs to process a very large amount of tokens.

3. subword tokenization

It relies on the principle that frequently used words should not be split into smaller subwords, but rare words should be decomposed into meaningful subwords.

(a) byte-level BPE (used in GPT-2)

2 Embedding Layer

After tokenization, there is an embedding layer to convert each token into a vector of dimension d . Assume there are $|V|$ tokens in the vocabulary, the embedding E can be viewed as a matrix of shape $|V| \times d$. The input sequence of N tokens can be represented using **one-hot embedding**, multiplying by the embedding matrix E , we can get an input matrix X of shape $N \times d$.

To represent the position of each token in the input sequence, we can combine the token embedding described above with **positional embedding**. One example of positional embedding is **absolute position**. We can have a positional embedding matrix E_{pos} of shape $N \times d$, the first row means the embedding for position 1. This position matrix can be learned along with other parameters during model training. In this case the final embedding for the first token is the sum of the token embedding and the position embedding for position 1.

3 Attention Layers

High level summary:

When a sentence is parsed in, we have a static embedding for each word in this sentence. But in reality, the meaning of a word in a sentence depends on the context (other words in the sentence). Attention layers modify the representation of words in a sentence by paying special attention to other words in the sentence.

3.1 Single (Masked, Self-)Attention Head

A single attention head uses **query**, **key** and **value** matrices, denoted by W^Q, W^K, W^V . For a d dimensional input vector $\mathbf{x}$, we can project $\mathbf{x}$ into three different vectors (**query**, **key** and **value**) using above matrices

$$\mathbf{q} = \mathbf{x}W^Q \quad \mathbf{k} = \mathbf{x}W^K \quad \mathbf{v} = \mathbf{x}W^V$$

Here, W^Q is of shape $d \times d_k$, W^K of shape $d \times d_k$, W^V of shape $d \times d_v$.

The basic idea is to represent token embedding $\mathbf{x}_i$ using prior tokens $\mathbf{x}_j$ with $j \leq i$. The basis is $\mathbf{v}'_j$ s, and the weight is the similarity between $\mathbf{q}_i$ and $\mathbf{k}'_j$ s. To be more precise,

$$\mathbf{q}_j = \mathbf{x}_j W^Q \quad \mathbf{k}_j = \mathbf{x}_j W^K \quad \mathbf{v}_j = \mathbf{x}_j W^V \quad \text{for all } j \leq i$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \quad \text{Notice the score is not symmetric, i.e. } \text{score}(\mathbf{x}_i, \mathbf{x}_j) \neq \text{score}(\mathbf{x}_j, \mathbf{x}_i)$$

$$\alpha_{i,j} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \text{for all } j \leq i. \text{ Normalization step}$$

$$\mathbf{head}_i = \sum_{j \leq i} \alpha_{i,j} \mathbf{v}_j$$

$$\mathbf{a}_i = \mathbf{head}_i W^O \quad \text{To make the final output be dim } d, W^O \text{ is of shape } d_v \times d$$

Example 3.1. One special example, W^Q, W^K and W^V are identity matrices of shape $d \times d$, $\text{score}(\mathbf{x}_i, \mathbf{x}_j)$ is the dot product, which can represent the similarity of two vectors.

3.2 Multi-head Attention

In the section above, there is only one attention head. There can be A attention heads sitting in parallel within the same transformer block. The intuition is that each head might be attending to the context for different purposes: heads might be specialized to represent different linguistic relationships between context elements and the current token, or to look for particular kinds of patterns in the context.

To be more precise, there are A sets of matrices $\{W^{Q_c}, W^{K_c}, W^{V_c}\}_c$. For a given input vector $\mathbf{x}_i$, the output for the multi-head attention is the concatenate of each $\mathbf{head}_i^c$.

$$\mathbf{head}_i = \mathbf{head}_i^1 \oplus \dots \oplus \mathbf{head}_i^c \oplus \dots \oplus \mathbf{head}_i^A$$

We use notation

$$\mathbf{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \dots, \mathbf{x}_{i-1}]) = \mathbf{head}_i W^O$$

The output for the multi-head attention is still of dimension d .

4 Transformer Blocks

A transformer block is a **residue stream**, consisting of several parts: **attention layer**, **feedforward layer** and **layer norm**.

1. Feedforward layer

It is a fully-connected 2-layer network, i.e. one hidden layer with ReLU as the activation function. ($\text{ReLU}(x) = \max(x, 0)$). It is common to make the dimensionality d_f of the hidden layer of the feedforward network be larger than the model dimensionality d (input dimension).

$$\mathbf{FF}(\mathbf{x}) = \text{ReLU}(\mathbf{x}W_1 + \mathbf{b}_1)W_2 + \mathbf{b}_2 \quad \in \mathbb{R}^{1 \times d} \quad \text{with } W_1 \in \mathbb{R}^{d \times d_f} \text{ and } W_2 \in \mathbb{R}^{d_f \times d}$$

2. Layer norm

It is common to add a layer norm in neural network to avoid overflow of gradients. Here we can apply the same trick. In the standard implementation of layer norm, there are two learnable parameters, γ and β , representing gain and offset values. The normalization layer is

$$\mathbf{LN}(\mathbf{x}) = \gamma \frac{\mathbf{x} - \mu}{\sigma} + \beta \quad \text{with } \mu \text{ being all coordinates of vector } \mathbf{x}, \text{ and } \sigma \text{ the standard derivation}$$

4.1 Residue Stream

The attention and the feedforward layers are the representation of the residue, not the input itself. To be more precise, given the input for i th token $\mathbf{x}_i$, the output after the whole transformer block is $\mathbf{h}_i$, then

$$\begin{aligned}
 \mathbf{t}_j^1 &= \mathbf{LN}(\mathbf{x}_j) \quad \text{for all } j \leq N && \text{normalize the inputs} \\
 \mathbf{t}_i^2 &= \mathbf{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{t}_1^1, \dots, \mathbf{t}_{i-1}^1]) && \text{attention layer} \\
 \mathbf{t}_i^3 &= \mathbf{x}_i + \mathbf{t}_i^2 && \text{relation with other tokens added as a residue} \\
 \mathbf{t}_i^4 &= \mathbf{LN}(\mathbf{t}_i^3) && \text{normalize the inputs} \\
 \mathbf{t}_i^5 &= \mathbf{FF}(\mathbf{t}_i^4) && \text{feedforward layer} \\
 \mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3 = \mathbf{x}_i + \mathbf{t}_i^2 + \mathbf{t}_i^5 && \text{FF layer output added as a residue}
 \end{aligned}$$

Why do we use **residue stream**?

- $\mathbf{x}_i$ carries everything learned so far, the attention layer and feedforward layer just correct it. If only the attention layer output is passed down to next layer, then next layer need to re-encode all info again, which is inefficient.
- **Gradient flow.** If there is a residue $\frac{\partial(\mathbf{h})}{\partial(\mathbf{x})} = \mathbf{I} + *$, there is always a gradient path. If not, if the gradient is initialized poorly, the gradient can be very slow or overflow.
- Intuitively, attention is more like a **contextual mixing**, representing the other info added from the context, not a full representation.
- With residue, it is easier to preserve the long-range features. Deeper layers can still access early features.

4.2 Matrix Representation of Transformer Block

We summarize the structure of transformer block in terms of matrix. Notation lists:

$$\begin{aligned}
 X &\in \mathbb{R}^{N \times d} && \text{input matrix} \\
 N &&& \text{number of tokens in the input sequence} \\
 d &&& \text{embedding dimension} \\
 A &&& \text{number of heads in the attention layer} \\
 W_c^K &\in \mathbb{R}^{d \times d_k} && \text{key matrix in attention head } c \\
 d_k &&& \text{dimension of the key vector, here we assume it is the same across all attention heads} \\
 W_c^Q &\in \mathbb{R}^{d \times d_k} && \text{query matrix in attention head } c, \text{ same shape as the key matrix} \\
 W_c^V &\in \mathbb{R}^{d \times d_v} && \text{value matrix in attention head } c \\
 d_v &&& \text{dimension of the value vector, all the same across all attention heads} \\
 W^O &\in \mathbb{R}^{Ad_v \times d} && \text{output matrix, making sure the output of attention heads is of the same shape as the input}
 \end{aligned}$$

The multi-head attention is

$$Q_c = XW_c^Q = \begin{pmatrix} \mathbf{q}_1 \\ \vdots \\ \mathbf{q}_N \end{pmatrix} \in \mathbb{R}^{N \times d_k} \quad K_c = XW_c^K = \begin{pmatrix} \mathbf{k}_1 \\ \vdots \\ \mathbf{k}_N \end{pmatrix} \in \mathbb{R}^{N \times d_k} \quad V_c = XW_c^V = \begin{pmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_N \end{pmatrix} \in \mathbb{R}^{N \times d_v}$$

query, key, value vectors

$$\frac{1}{\sqrt{d_k}} Q_c K_c^T = (\mathbf{q}_i \mathbf{k}_j^T)_{1 \leq i, j \leq N} \quad \text{need to mask the future i.e. attention for } \mathbf{x}_i \text{ does not use tokens after } i$$

$$\alpha_c = \text{Softmax}(\text{masked}(\frac{1}{\sqrt{d_k}} Q_c K_c^T)) \in \mathbb{R}^{N \times N} \quad \text{mask the strictly upper diagonal elements into } -\infty$$

Softmax is applied row-wise;

because of the mask, the weight matrix α_c is a lower diagonal matrix with strictly upper diagonal elements 0

$$\mathbf{head}_c = \text{Causal Self-Attention}(Q_c, K_c, V_c) = \alpha_c V_c \in \mathbb{R}^{N \times d_v}$$

$$\text{Multi-Head Attention}(X) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \cdots \oplus \mathbf{head}_A) W^o \in \mathbb{R}^{N \times d}$$

The residue stream transformer block has output

$$\mathbf{O} = \mathbf{X} + \mathbf{MultiHeadAttention}(\mathbf{LN}(X)) \quad \text{multi-head causal attention layer}$$

$$\mathbf{H} = \mathbf{O} + \mathbf{FF}(\mathbf{LN}(\mathbf{O})) \quad \text{Feedforward layer}$$

5 Language Model Head

For each token input $\mathbf{x}_i \in \mathbb{R}^{1 \times d}$, after layers of transformer blocks the output is $\mathbf{h}_i \in \mathbb{R}^{1 \times d}$. But in the end we need to have token as output, thus we need a language model head. In details, we can apply the unembedding matrix $\mathbf{E}^T \in \mathbb{R}^{d \times |V|}$ (the transverse of the embedding matrix $\mathbf{E}$), then apply the softmax function to get the probability distribution for each token.

$$\mathbf{p}_i = \text{Softmax}(\mathbf{h}_i \mathbf{E}^T) \in \mathbb{R}^{1 \times |V|}$$

To get the final token, there are multiple different ways of sampling.

6 Different Architectures of Transformer Models

There are three main types of transformer models: encoder-only/masked language model, decoder-only/casual/auto-regressive language model, encoder-decoder models. Here these words are not perfectly synonymous, but in practice they mean the same thing.

6.1 decoder-only/casual/auto-regressive language model

By usage, this category of models is **autoregressive** models. Example models are **GPT**, **LLaMA**. To be more precise, it represents models with

- **Self-attention only** Query, key and value vectors are all from one input sentence.
- **Causal attention** The upper triangular mask (as mentioned above) is applied in the attention layer. The token can only learn from previous tokens, not from future tokens.
- **Autoregression generation** The training objective is to find the next token y_t given previous tokens $y_1, \dots, y_{t-1}$.

6.1.1 model training

The detailed model structure is described in previous section. In this section, we emphasize the model training. For the auto-regressive model, the output vector through the model for x_i represents the token probability distribution $\{p(w|x_{<i})\}_{w \in V}$.

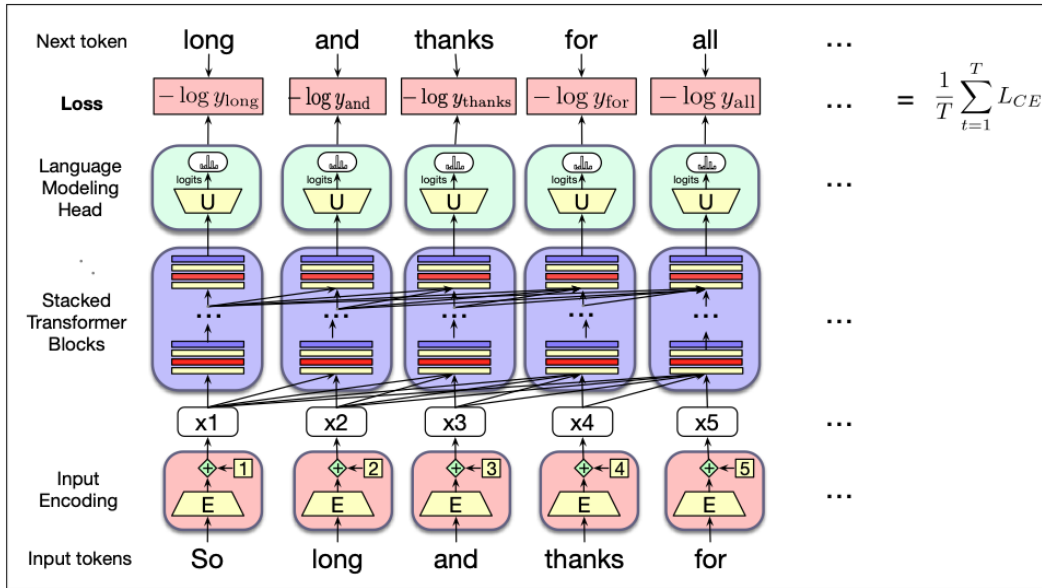


Figure 8.16 Training a transformer as a language model.

Figure 1:

The training objective is to maximize $p(x) = \prod_t p(x_t | x_{<t})$ with x being an input training token sequence. We can think of the objective function from either the **cross-entropy** or **maximizing the likelihood** perspective.

Below is a detailed picture of the loss function and the training process. This picture is from [section 8.7](#) of the book. [1].

6.2 encoder-only/masked language model

By usage, this category of models is **representation** models. Example models are **BERT**, **RoBERTa**. To be more precise, it represents models with

- **Self-attention only**
- **Bidirectional attention** There is no mask in the attention layer. A token can learn information from future tokens.
- **Non-generative objectives** For example, it can be trained via masked language modeling.

6.2.1 model training

The model structure is basically the same as the one described in previous sections. The main difference is that the model uses bidirectional attention which allows the information of future tokens.

The model usage is different, so is the training objective. Here we introduce **Masked Language Model (MLM)** (predicting words from surrounding contexts) and **Next Sentence Prediction (NSP)** (predicting whether a pair of sentences consists of an actual pair of adjacent sentences from the training corpus or a pair of unrelated sentences), which are the training objectives in **BERT** model.

- **Masked Language Model**

The model is presented with a series of sentences from the training corpus in which a percentage of tokens (15% in the **BERT** model) have been randomly chosen to be manipulated by the **masking** procedure. The MLM training goal is to predict these masked tokens. The masking procedure is defined in details as follows:

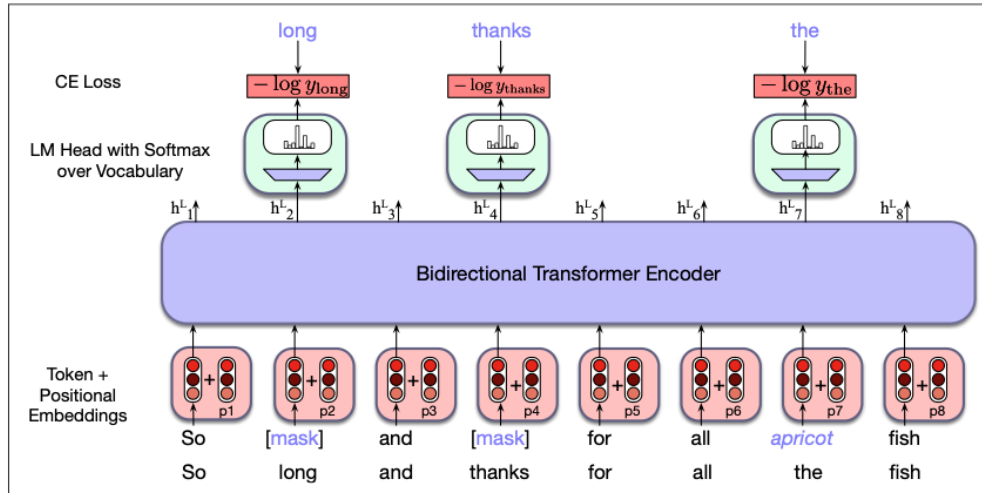


Figure 10.3 Masked language model training. In this example, three of the input tokens are selected, two of which are masked and the third is replaced with an unrelated word. The probabilities assigned by the model to these three items are used as the training loss. The other 5 tokens don't play a role in training loss.

1. 80% of the time: The token is replaced by a special token [MASK].
If we replace **all** with [MASK], the model might only predict when it sees [MASK] token, but we want it always predict.
2. 10% of the time: The token is replaced with another token, randomly sampled from the vocabulary based on token uni-gram probabilities.
This is to increase the model's robustness. It forces model to avoid blindly copying of visible tokens, thus to build robust contextual embeddings.
3. 10% of the time: The token is left unchanged.
This makes model to predict even when the token is already visible; This teaches the model that not every prediction is signaled by [MASK] and the representation must be useful when the token is already present.

Now, the output h_i for the transformer block of x_i represents the probability distribution $\{p(w|x_{\neq i})\}_{w \in V}$. The loss is the average of the **Cross Entropy** of all the **masked** tokens. Note that the other unmasked words play no role in the loss function, so in this sense **BERT** and its descendants are inefficient; only 15% of the input samples in the training data are actually used for training weights.

The training details are illustrated in this picture. This picture is from [section 10.2.1](#) of the book [\[1\]](#).

• Next Sentence Prediction

In this task, the model is presented with pairs of sentences and is asked to predict whether each pair consists of an actual pair of adjacent sentences from the training corpus or a pair of unrelated sentences. In **BERT**, 50% of the training pairs consisted of positive pairs, and in the other 50% the second sentence of a pair was randomly selected from elsewhere in the corpus. The **NSP** loss is based on how well the model can distinguish true pairs from random pairs.

To prepare the training data, the **BERT** introduces two special tokens [CLS] and [SEP]. We use [Sentence 1] and [Sentence 2] to represent the two lists of tokens for the pair of training sentences. The model input tokens will be the concatenate of the these tokens as below

$$[\text{CLS}] + [\text{Sentence 1}] + [\text{SEP}] + [\text{Sentence 2}]$$

The output for [CLS] token after the transformer block h_{CLS} is expected to be a representation of the pair of sentences.

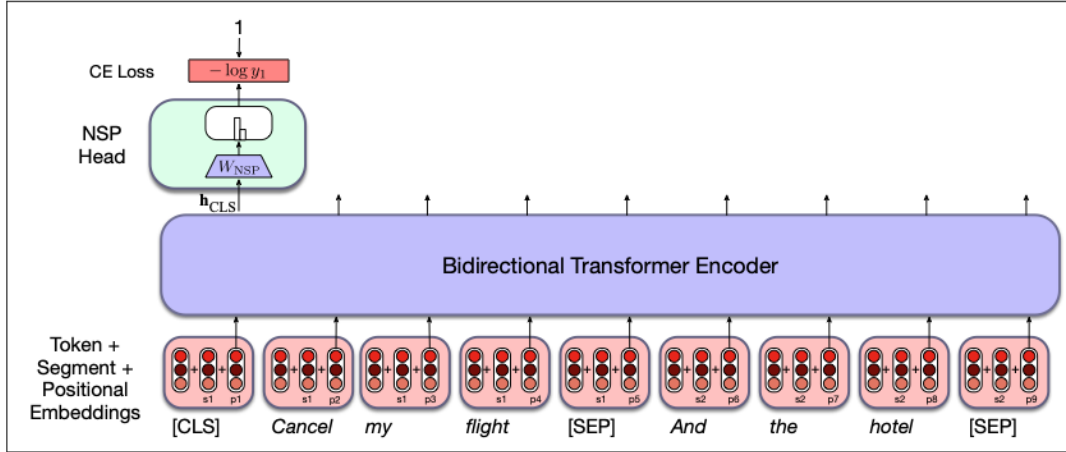


Figure 10.4 An example of the NSP loss calculation.

The model will have a special **NSP** head to convert above representation $\mathbf{h}_{CLS}$ to a probability prediction of this two-class classification problem. To be more precise, we use a learned weight matrix $W_{NSP} \in \mathbb{R}^{d \times 2}$ to get the probability output $\mathbf{y}_{CLS}$ by

$$\mathbf{y}_{CLS} = \text{softmax}(\mathbf{h}_{CLS} W_{NSP})$$

Cross entropy is used to compute the **NSP** loss for each sentence pair presented to the model. The figure below illustrates the overall **NSP** training setup. This picture is from [section 10.2.2](#) of the book [1].

[CLS] representation is useful in the fine-tuning for classification problems, since it is a representation of the whole sentence.

- **BERT** training

The loss function for **BERT** is a combination of above two training objectives.

6.3 encoder-decoder models

By usage, this category of models is **sequence-to-sequence** models. Example models are **T5**, **BART**. To be more precise, it represents models with

- **Encoder: bidirectional self-attention**
- **Decoder: causal self-attention + cross-attention** Cross-attention
- **Seq2Seq generation** For example, it can be trained for machine translation; given a source sentence $x_1, \dots, x_N$, and part of the target sequence $y_1, \dots, y_{t-1}$, we want to generate the next token y_t .

6.3.1 cross attention

6.3.2 model training

We discuss the training objective in the use case of machine translation.

6.4 Terminology Caveat

References

- [1] Daniel Jurafsky and James H. Martin. (2025). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3rd edition. [Online manuscript](#) released August 24, 2025.